1. **Define computer networking. Why is networking important in Java applications?**
   Computer Networking is the practice of connecting computers and devices together to share resources, data, and services. This includes the Internet, LAN, WAN, etc.
   Importance in Java:

- Enables communication between distributed systems.
- Allows Java applications to access web services, REST APIs, databases, etc.
- Used in chat apps, file transfers, multiplayer games, etc.

2. **Differentiate between TCP and UDP protocols. Where would you use each in Java?**

Feature	**TCP**	**UDP**
Type	Connection-oriented	Connectionless
Reliability	Ensures delivery	No guarantee of delivery
Speed	Slower due to error checking	Faster but unreliable
Use Case	Email, File transfer, Web	Video streaming, Online gaming
Java Classes	Socket, ServerSocket	DatagramSocket, DatagramPacket

3. **What is the purpose of the InetAddress class in Java?**

   The InetAddress class in Java, part of the java.net package, represents **IP addresses** (both IPv4 and IPv6) and provides methods for working with **hostnames and IP address resolution**. It acts as a bridge between Java applications and the underlying **networking layer** of the operating system.

   Main Purposes of InetAddress

1. **IP Resolution (Domain Name → IP):**
   Convert a domain name (like "google.com") into its IP address.
2. **Hostname Resolution (IP → Domain Name):**
   Resolve an IP address into a human-readable domain name.
3. **Check Reachability:**
   Test if a host is reachable (similar to ping).
4. **Local Address Information:**
   Retrieve the IP address and hostname of the local machine.

   Key Methods in InetAddress

Method	**Description**
getByName(String host)	Returns the InetAddress object for a given host name or IP address.

Method	Description
getHostName()	Returns the host name for this IP address.
getHostAddress()	Returns the IP address as a string.
isReachable(int timeout)	Checks if the address is reachable within a timeout (in ms).
getLocalHost()	Returns the InetAddress of the local machine.

4. Explain the role of the Socket and ServerSocket classes in Java networking.

- ServerSocket: Listens on a port and waits for client connections.
- Socket: Used by the client to connect to the server or by the server to communicate with the client after connection.
  code
  // Server
  ServerSocket serverSocket = new ServerSocket(5000);
  Socket socket = serverSocket.accept(); // waits for client

  // Client
  Socket socket = new Socket("localhost", 5000);

5. **What is DatagramSocket? How does it help in distributed computing?**
   The DatagramSocket class in Java is part of the java.net package and is used for **UDP (User Datagram Protocol)** based network communication. Unlike TCP sockets (Socket and ServerSocket), which require a connection to be established between the client and server, a DatagramSocket is **connectionless**. It sends and receives data in the form of **independent packets** called **datagrams**.

How DatagramSocket Works:

- It uses DatagramPacket to wrap the data.
- There is **no handshaking**, meaning data can be sent without waiting for a connection.
- It is **lightweight and faster** than TCP but does **not guarantee delivery, order, or reliability**.

How It Helps in Distributed Computing:

In distributed computing, multiple systems work together over a network to solve problems or share resources. DatagramSocket helps in such environments by:

1. **Enabling Lightweight Communication:**
   - Ideal for scenarios where **quick, one-way communication** is needed (e.g., sensor updates, heartbeat signals).
2. **Reducing Overhead:**
   - Since it doesn't establish a connection, it reduces **setup time and memory usage**, which is useful when communicating with many distributed nodes.

3. **Supporting Broadcast and Multicast:**
    - Datagram sockets can be used to **send the same data to multiple machines** in a network, which is helpful in distributed systems like discovery services.
4. **Fault Tolerance & Simplicity:**
    - Distributed systems often expect unreliable environments. DatagramSocket fits naturally when the system is built to **tolerate data loss** or retries.

Example (Sending a Datagram):

```
DatagramSocket socket = new DatagramSocket();
String msg = "Hello, Node!";
InetAddress address = InetAddress.getByName("192.168.1.100");
byte[] buffer = msg.getBytes();
DatagramPacket packet = new DatagramPacket(buffer, buffer.length, address, 8080);
socket.send(packet);
```

6. Explain the concept of Java RMI. How does it help in building distributed computing applications?

Java RMI (Remote Method Invocation) allows an object on one JVM to invoke methods on an object in another JVM, as if they were local.

How RMI Supports Distributed Computing:

1. **Remote Method Invocation**: Clients can invoke methods on remote server objects transparently, just like calling local methods.
2. **Object Passing**: Java RMI supports both primitive data types and objects (via serialization), enabling complex data exchange across machines.
3. **Modular Architecture**: The application logic can be split into client-side and server-side, allowing different parts of the system to be updated or scaled independently.
4. **Encapsulation**: Business logic remains on the server; clients only use exposed methods, improving security and code reuse.

Key Components of RMI:

Component	Description
Remote Interface	Defines the methods that can be called remotely.
Remote Object	Implements the remote interface and runs on the server.
RMI Registry	A naming service that allows clients to locate remote objects.
Client	Looks up the remote object and invokes its methods.

Benefits in Distributed Systems:

- Encourages **separation of concerns** between UI and business logic.
- Simplifies **network programming** compared to low-level socket code.
- Facilitates **load distribution** and efficient resource usage.
- Can be integrated with **databases, filesystems, and other backend services**.

7. Write a simple program of RMI implementation with a database(paste the screen shot of code and output)

DatabaseService.java

```
import java.rmi.*;

public interface DatabaseService extends Remote {
   String getStudentName(int rollNo) throws RemoteException;
}
```

DatabaseServiceImpl.java

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.sql.*;


public class DatabaseServiceImpl extends UnicastRemoteObject implements DatabaseService {
   Connection conn;


   protected DatabaseServiceImpl() throws RemoteException {
      try {
         conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root",
"password");
      } catch (SQLException e) {
         e.printStackTrace();
      }
   }


   public String getStudentName(int rollNo) throws RemoteException {
      try {
         PreparedStatement ps = conn.prepareStatement("SELECT name FROM students WHERE
roll_no=?");
         ps.setInt(1, rollNo);
         ResultSet rs = ps.executeQuery();
         if (rs.next()) return rs.getString("name");
      } catch (SQLException e) {
```

```
            e.printStackTrace();
        }
        return "Not Found";
    }
}
```

RMIServer.java

```
import java.rmi.*;
import java.rmi.registry.*;

public class RMIServer {
    public static void main(String args[]) {
        try {
            DatabaseServiceImpl obj = new DatabaseServiceImpl();
            LocateRegistry.createRegistry(1099);
            Naming.rebind("DBService", obj);
            System.out.println("RMI Server is ready.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

RMIClient.java

```
import java.rmi.*;
public class RMIClient {
    public static void main(String[] args) {
        try {
            DatabaseService stub = (DatabaseService) Naming.lookup("rmi://localhost/DBService");
            System.out.println("Student Name: " + stub.getStudentName(101));
        } catch (Exception e) {
            e.printStackTrace();
        }
```

```
    }
}
```

Server Console Output (RMIServer.java)

RMI Server is ready.

Client Console Output (RMIClient.java)

Student Name: Tara

If no such roll number exists:

Student Name: Not Found

8. How does the UnicastRemoteObject class help in creating remote objects in Java RMI?

In Java RMI, the UnicastRemoteObject class plays a key role in **making a remote object available to accept incoming remote method calls**. It is a part of the java.rmi.server package and provides the functionality required to export a remote object so that it can be accessed over the network.

How It Helps in Creating Remote Objects:

There are **two main ways** to use UnicastRemoteObject:

1. Extending the class:

Most commonly, the remote implementation class extends UnicastRemoteObject, and the export happens automatically in the constructor.

```
public class MyRemoteImpl extends UnicastRemoteObject implements MyRemoteInterface {
    public MyRemoteImpl() throws RemoteException {
        super(); // Exports the object
    }

    public String sayHello() throws RemoteException {
        return "Hello from server!";
    }
}
```

2. Using exportObject() method manually:

You can also **manually export** the object without extending the class:

```
MyRemoteInterface stub = (MyRemoteInterface)
    UnicastRemoteObject.exportObject(new MyRemoteImpl(), 0);
```

This approach gives more flexibility, especially when your class already extends another class.

9. Outline the steps needed to create an RMI server and client to demonstrate a remote method call.

Creating an RMI (Remote Method Invocation) application in Java involves setting up both a **server** (that hosts the remote object) and a **client** (that calls the remote method).

Step 1: Define a Remote Interface

- Create a Java interface that **extends java.rmi.Remote**.
- Declare the remote methods, which must **throw RemoteException**.

```
import java.rmi.*;

public interface MyService extends Remote {
   String greet(String name) throws RemoteException;
}
```

Step 2: Implement the Remote Interface

- Create a class that **implements** the remote interface.
- This class should either **extend UnicastRemoteObject** or be exported using UnicastRemoteObject.exportObject().

```
import java.rmi.server.UnicastRemoteObject;

import java.rmi.RemoteException;

public class MyServiceImpl extends UnicastRemoteObject implements MyService {
   public MyServiceImpl() throws RemoteException {
      super();
   }


   public String greet(String name) {
      return "Hello, " + name + "!";
   }
}
```

Step 3: Start and Register the RMI Server

- Create an instance of the remote object.
- Start the **RMI registry** programmatically using LocateRegistry.createRegistry(port).
- Bind the remote object to the registry using Naming.rebind() or Registry.bind().

```
import java.rmi.Naming;

import java.rmi.registry.LocateRegistry;
```

```
public class RMIServer {
    public static void main(String[] args) {
        try {
            MyService service = new MyServiceImpl();
            LocateRegistry.createRegistry(1099); // Start RMI registry
            Naming.rebind("MyService", service); // Bind service
            System.out.println("RMI Server is running...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Step 4: Create the RMI Client

- Use Naming.lookup() to retrieve the remote object from the RMI registry.
- Call the remote method as if it were a local method.

```
import java.rmi.Naming;
public class RMIClient {
    public static void main(String[] args) {
        try {
            MyService service = (MyService) Naming.lookup("rmi://localhost/MyService");
            String message = service.greet("Roshan");
            System.out.println("Server Response: " + message);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Step 5: Compile the Code

Compile all the .java files:

```
javac *.java
```

If using Java 8 or higher, you don't need to generate stubs with rmic.

Step 6: Run the Server and Client

1. **Start the server:**

java RMIServer

2. **In another terminal, run the client:**

java RMIClient

Output

Server: RMI Server is running...

Client: Server Response: Hello, Tara!

10. What is the role of the RMI Registry? How do you bind and look up remote objects?

The **RMI Registry** is a built-in naming service provided by Java that allows clients to **locate and obtain references to remote objects** hosted on a server. It acts as a **directory** where remote objects are registered with a unique name, enabling remote method invocation across different JVMs.

Role of RMI Registry:

- Acts as a **central registry** for storing references to remote objects.
- Allows **clients to discover remote objects** using a name (like a URL).
- Helps decouple the client from the server, improving modularity and flexibility.

Binding a Remote Object to Registry:

The server binds (registers) a remote object using:

Naming.rebind("ServiceName", remoteObject);

This associates the name "ServiceName" with the given remote object instance.

Looking Up a Remote Object (Client Side):

The client retrieves the remote object reference using:

RemoteInterface stub = (RemoteInterface) Naming.lookup("rmi://localhost/ServiceName");

This fetches the remote object registered under "ServiceName" on the server running at localhost.

11. Explain how RMI can be integrated with a database backend to provide remote access to data.

Java RMI can be effectively integrated with a database backend (e.g., MySQL, Oracle) to allow **clients to perform database operations remotely** by calling methods hosted on an RMI server.

How It Works:

1. **Server-side JDBC Integration**:
    - The RMI server maintains a **JDBC connection** with the database.
    - It implements methods like getUserById(int id) or getStudentDetails() to interact with the database.
2. **Exposing Methods via RMI Interface**:
    - These database access methods are declared in the **remote interface**.
    - Clients can call these methods as if they were local, but they execute on the server.
3. **Client Calls, Server Executes**:
    - The client sends a method call request.
    - The server **queries the database**, processes the result, and returns data back to the client.

Example Use Case:

Remote Interface:

```
public interface DatabaseService extends Remote {
   String getStudentName(int rollNo) throws RemoteException;
}
```

Server Implementation:

```
public String getStudentName(int rollNo) throws RemoteException {
   PreparedStatement ps = conn.prepareStatement("SELECT name FROM students WHERE roll_no=?");
   ps.setInt(1, rollNo);
   ResultSet rs = ps.executeQuery();
   if (rs.next()) return rs.getString("name");
   return "Not Found";
}
```